



Third Year Engineering

18BTCS601-Design and Analysis of Algorithms

Class - T.Y. (SEM-I)

Unit - I

Fundamentals of Algorithm

AY 2024-2025 SEM-I





Unit-I **Syllabus**

- **What is algorithm, Fundamentals of algorithmic problem solving**
- **The Analysis Framework: Measuring an Input's Size, Units for Measuring Running Time, Orders of Growth, Worst-Case, Best-Case, and Average-Case Efficiencies**
- **Asymptotic notations: Asymptotic complexities calculations for given function**
- **Mathematical Analysis of Non recursive Algorithms: Finding the value of largest element in the list, element uniqueness problem**
- **Mathematical Analysis of Recursive Algorithms: Factorial function**
- **Master Theorem: Solving recurrence relations using master theorem**



What is algorithm

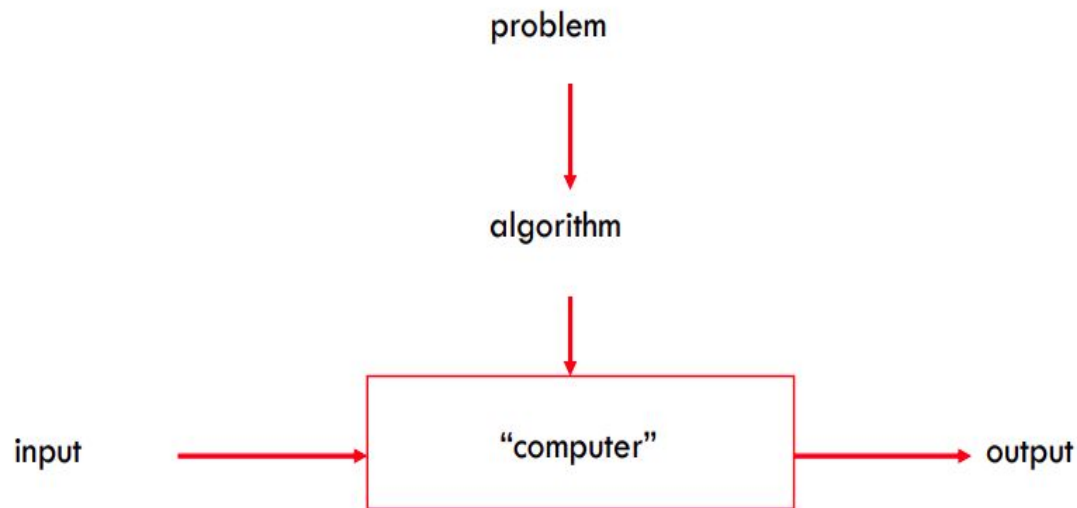
A finite set of instruction that specify a sequence of operations to be carried out in order to solve a specific problem or class of problems is called an algorithm.

or

Algorithm is defined as a step by step procedure to perform a specific task within finite number of steps



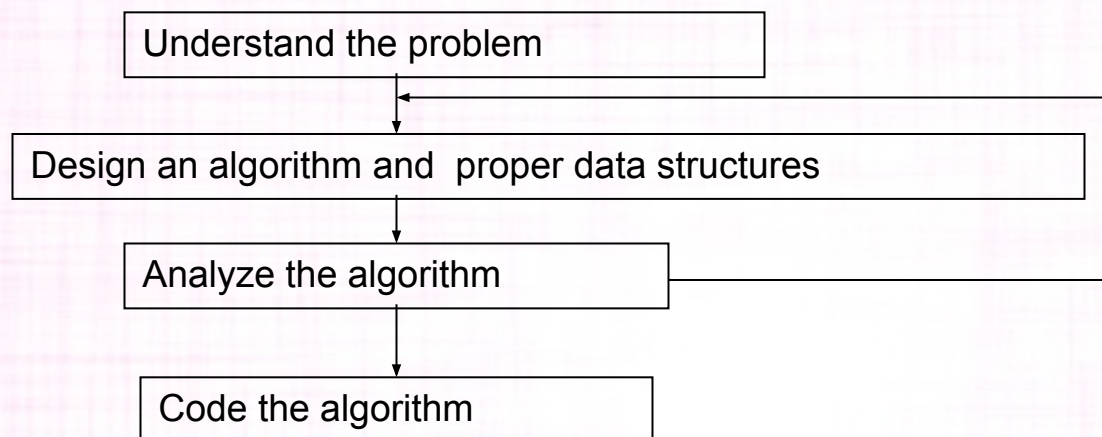
What is algorithm



Algorithmic solution



Algorithmic Problem Solving



- Ascertaining the Capabilities of a Computational Device
- Choosing between Exact and Approximate Problem Solving
- Deciding on Appropriate Data Structures
- Algorithm Design
- Algorithm Analysis
- Coding





PSEUDOCODE

Pseudocode (pronounced SOO-doh-kohd) is a detailed yet readable description of what a computer program or algorithm must do, expressed in a formally-styled natural language rather than in a programming language.

It is sometimes used as a detailed step in the process of developing a program. It allows programmers to express the design in great detail and provides programmers a detailed template for the next step of writing code in a specific programming language.



PSEUDOCODE to find the sum of numbers

- `int N, sum = 0, i = 1.`
- Input N from user.
- do.
- `sum = sum + i.`
- `i = i + 1.`
- Repeat step 3 to 5 till $i \leq N$.
- Display the value of sum.





Fundamentals of algorithmic problem solving

Every algorithm should have the following five characteristic features:

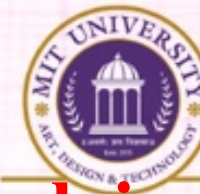
- a) Input**
- b) Output**
- c) Finiteness**
- d) Definiteness**
- e) Effectiveness**





Fundamentals of algorithmic problem solving

- a) Input:** An algorithm may have one or more inputs. The inputs are taken from a specified set of subjects. The input pattern may be texts, images or any type of files.
- b) Output:** An algorithm may have one or more outputs. Output is basically a quantity which has a specified relation with the input.
- c) Finiteness:** An algorithm should terminate after a countable number of steps. In some cases the repetition of steps may be larger in number. If a procedure is able to resolve in finite number of execution of steps, then it is referred to be computational method.



Fundamentals of algorithmic problem solving

d) Definiteness: Each step of an algorithm must be precisely defined. Due to the lack of understandability one may think that the step might be lacking definiteness. Therefore in such cases mathematically expressions are written, so that it resembles the instruction of any computer language.

e) Effectiveness: An algorithm is generally expected to be effective. Means the steps should be sufficiently basic, so that it may be possible for a man to resolve them.



Expectation from an algorithm

Correctness: Produced by the algorithm should correct.

Approximation :Exact solution is not found, but near optimal solution can be found out

Generality: Algorithm applies to set of standard inputs.

Less resource usage: Algorithms should use less resources (time and space).



What is Computer algorithm?

"A set of steps to accomplish or complete a task that is described precisely enough that a computer can run it".

Described precisely: very difficult for a machine to know how much water, milk to be added etc.

Example: Tea making algorithm.(very difficult for a machine to know how much water, milk to be added etc.)



What is Computer algorithm?

These algorithms run on computers or computational devices.

For example: GPS in our smartphones. GPS uses shortest path algorithm.

Online shopping uses cryptography which uses RSA algorithm





The Analysis Framework:

Analysis of algorithms means investigation of an algorithm's efficiency with respect to two resources: running time and memory space.

i.e, *Analysis* refers to the task of determining how much computing time and storage an algorithm requires.

Space complexity of an algorithm is the amount of memory it needs to run to completion.

Time complexity of an algorithm is the amount of computer time it needs to run to completion.

Performance evaluation can be divided into two major phases:

- a priori estimates (Performance analysis)
- a posteriori testing (Performance measurements)



The Analysis Framework:

Measuring an Input's Size:

- Time complexity depends on the number of inputs also. i.e, the running time of an algorithm increases with the input size.

Example: It takes longer time to sort larger arrays, to multiply larger matrices and so on.

- Therefore, it is logical to investigate an algorithm's efficiency *as a function of some parameter n indicating the algorithm's input size.*
- In most cases, selecting parameter n is straight forward.

Example:

- ✓ For problems of *sorting, searching, finding largest element* etc., the size metric will be the *size of the list.*



The Analysis Framework:

- ✓ For problem of *computing the product of two $n \times n$ matrices*, the natural size measures are frequently used *matrix order n* and the *total number of elements N* in the matrices being multiplied.
 - The algorithm's efficiency will be qualitatively different depending on which of the two measures we use.
- ✓ The choice of the appropriate size metric can be influenced by operations of the algorithm.



Measuring an Input's Size:

Example: Consider a *spell-checking algorithm*:

- If the algorithm examines individual characters then the size measure will be the *number of characters* in the input.
- If the algorithm works by processing words, then the size measure will be the *number of words* in the input.



Units for Measuring Running Time:

- Some standard unit of time measurements – a second, a millisecond, and so on can be used to *measure the running time* of a program implementing the algorithm.
- The *obvious drawbacks* to above approach are:
 - dependence on the speed of a particular computer,
 - dependence on the quality of a program implementing the algorithm,
 - compiler used in generating the machine code,
 - difficulty of clocking the actual running time of the program.
- For measuring the algorithm's efficiency, we need to have a metric that does not depend on these hardware and software factors.



Units for Measuring Running Time:

To measure an algorithm's efficiency:

- One possible approach is to *count the number of times each of the algorithm's operations are executed*. This approach is both excessively difficult and usually unnecessary.
- Another approach is to identify the *basic operation(primitive operation)*, i.e., the operation contributing the most to the total running time, and compute the number of times the basic operation is executed on inputs of size n .

Example:

- *Sorting algorithms* works by comparing elements of a list being sorted with each other. For such algorithms, basic operation is a *key comparison*.
- *Matrix multiplication and polynomial evaluation* requires two arithmetic operations: multiplication and addition. On most computers, multiplication of two numbers takes longer time than addition. Hence, the basic operation considered is *multiplication*.



INPUT SIZE AND BASIC OPERATION EXAMPLES

Problem	Input Size Measure	Basic Operation
Search for a key in a list of n items	Number of items in the list, n	Key comparison
Sort the list of n items	Number of items in the list, n	Key comparison
Add two n by n matrices	Dimension of matrices, n	Addition



Units for Measuring Running Time:

- The established framework for the analysis of an algorithm's time efficiency suggests measuring time efficiency by counting the number of times the algorithm's basic operation is executed on input's of size n .
- Let C_{op} be the execution time of an algorithm's basic operation on a particular computer.
 $C(n)$ be the number of times this basic operation needs to be executed for this algorithm.

Then, the running time $T(n)$ can be estimated by

$$T(n) \approx C_{op} \cdot C(n)$$

Note: C_{op} is an approximation & $C(n)$ does not contain any information about operations that are not basic. Therefore, above formula can give a reasonable estimate of the algorithm's running time.



Units for Measuring Running Time:

Problem: Assuming $C(n) = \frac{1}{2} n (n - 1)$, how much longer will the algorithm run if we double its input size?

$$\text{Solution: } C(n) = \frac{1}{2} n (n - 1) = \frac{1}{2} n^2 - \frac{1}{2} n \approx \frac{1}{2} n^2$$

$$\text{Therefore, } \frac{T(2n)}{T(n)} \approx \frac{C_{\text{op}} \cdot C(2n)}{C_{\text{op}} \cdot C(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2} n^2} = 4$$

Note: The Efficiency analysis framework ignores multiplicative constants and concentrates on the basic operation count's order of growth for large-size inputs.



Order of Growth:

It is described by the highest degree term of the formula for running time. (Drop lower-order terms. Ignore the constant coefficient in the leading term.)

ORDERS OF GROWTH

- For $\text{GCD}(m, n)$ of two small numbers, it is not clear how much more efficient Euclid's algorithm is, compared to other two algorithms.
- It is only when we have to find the GCD of two large numbers (eg., $\text{GCD}(31415, 14142)$) that the difference in algorithm efficiencies becomes both clear and important.
- For large values of n , it is the function's order of growth that counts.



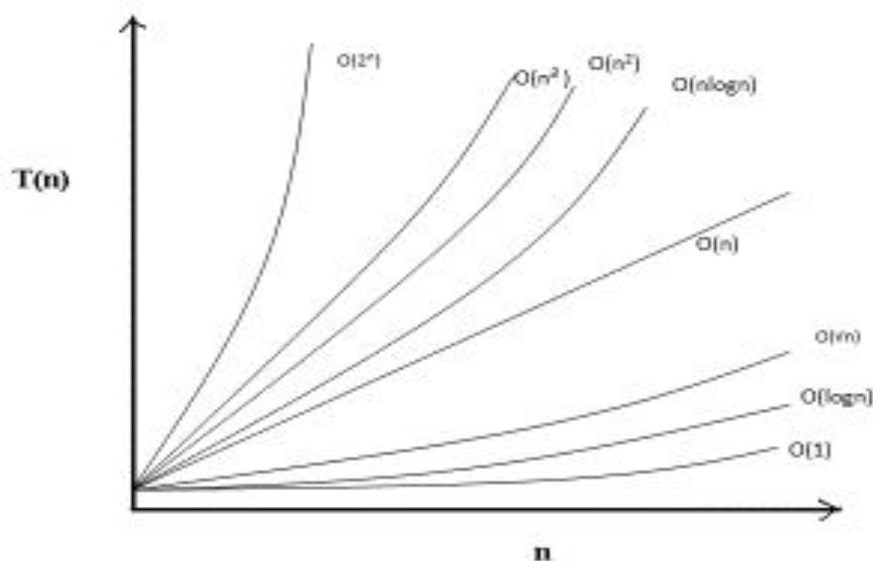
ORDERS OF GROWTH

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	3.3×10^1	10^2	10^3	10^3	3.6×10^6
10^2	6.6	10^2	6.6×10^2	10^4	10^6	1.3×10^{30}	9.3×10^{157}
10^3	10	10^3	1.0×10^4	10^6	10^9		
10^4	13	10^4	1.3×10^5	10^8	10^{12}		
10^5	17	10^5	1.7×10^6	10^{10}	10^{15}		
10^6	20	10^6	2.0×10^7	10^{12}	10^{18}		

Table: Values (some approximate) of several functions important for analysis of algorithms.



Common plots of $O()$





ORDERS OF GROWTH

- The function growing the slowest among these is the *logarithmic function*.
- A program implementing an algorithm with a *logarithmic basic-operation count* runs *practically* instantaneously on *inputs of all realistic sizes*.
- *Exponential function 2^n and the factorial function $n!$* grows so fast that their values become large even for small values of n .



Worst-Case, Best-Case and Average-Case Efficiencies:

- There are many algorithms for which running time depends not only on an input's size, but also on the specifics of a particular input.

ALGORITHM *SequentialSearch*($A[0\dots n-1]$, k)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0\dots n-1]$ and a search key k

//Output: The index of the first element of A that matches k or -1 if

// there are no matching elements.

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq k$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

- Running Time of this algorithm can be quite different for the same list of size n .



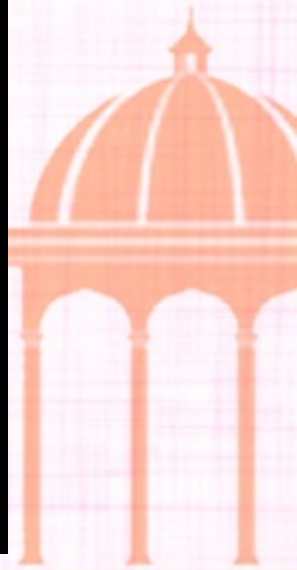
Worst-Case, Best-Case and Average-Case Efficiencies:

Worst – Case Efficiency:

- When there are no matching elements or the first matching element happens to be the last one in the list, the algorithm makes the largest number of key comparisons.

$$C_{\text{worst}}(n) = n$$

- The **Worst – Case Efficiency** of an algorithm is its efficiency for the worst – case input of size n , which is an input for which the algorithm runs the longest among all possible inputs of that size.
- Analyze the algorithm to see what kind of inputs yields the largest value of the basic operation's count $C(n)$ among all possible inputs of size n and then compute $C_{\text{worst}}(n)$.





Worst-Case, Best-Case and Average-Case Efficiencies:

Best – Case Efficiency:

- The **Best – Case Efficiency** of an algorithm is its efficiency for the best – case input of size n , which is an input for which the algorithm runs the fastest among all possible inputs of that size.
- Analyze the algorithm to see what kind of inputs yields the smallest value of the basic operation's count $C(n)$ among all possible inputs of size n and then compute $C_{\text{best}}(n)$.
- For Sequential search, best-case inputs are lists of size n with their first elements equal to a search key.

$$C_{\text{best}}(n) = 1 \quad \text{for successful search.}$$

$$C_{\text{best}}(n) = n \quad \text{for unsuccessful search.}$$



Worst-Case, Best-Case and Average-Case Efficiencies:

Average - Case Efficiency:

- Neither the worst-case analysis nor its best-case counterpart yields the necessary information about an algorithm's behavior on a typical or random input. This is provided by **Average-case analysis**.

Assumptions for Sequential Search:

- (a) The probability of a successful search is equal to P ($0 \leq P \leq 1$).
- (b) Probability of first match occurring at i^{th} position is same for every i .

Now, find the average number of key comparisons $C_{\text{avg}}(n)$ as follows:

- In case of successful search, the probability of the first match occurring in i^{th} position of the list is p/n for every i and number of comparisons is i .



Worst-Case, Best-Case and Average-Case Efficiencies:

Average - Case Efficiency:

- In case of unsuccessful search, the number of comparisons is n , with the probability of such a search being $(1-P)$.

$$C_{avg}(n) = \underbrace{[1 \cdot P + 2 \cdot P + \dots + i \cdot P + \dots + n \cdot P]}_{\text{For Successful search}} + \underbrace{n \cdot (1 - P)}_{\text{For Unsuccessful search}}$$

For Successful search

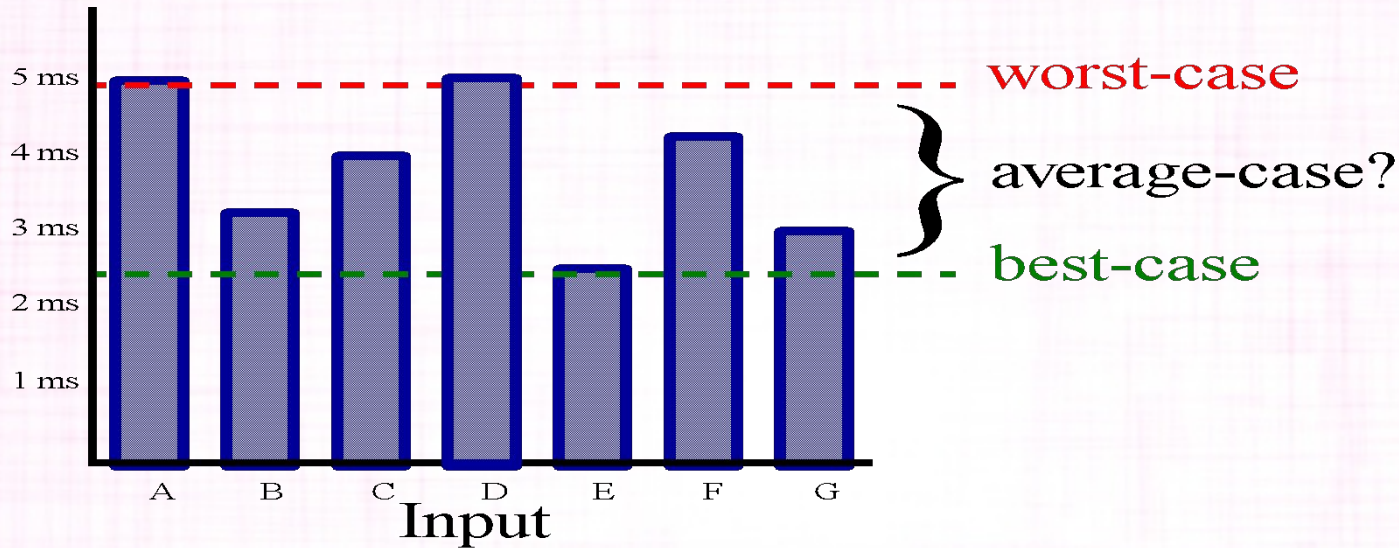
For Unsuccessful search

$$= P[1 + 2 + \dots + n] + n(1 - P)$$

$$= P \frac{n(n+1)}{2} + n(1 - P) = \frac{P(n+1)}{2} + n(1 - P)$$

If $P = 1$ (i.e., successful search) then $C_{avg}(n) = (n+1)/2$

If $P=0$ (i.e., Unsuccessful search) then $C_{avg}(n) = n$.



Suppose the program includes an *if-then* statement that may execute or not: ☐ variable running time

Typically algorithms are measured by their **worst case**





RECAPITULATION OF THE ANALYSIS FRAMEWORK

- ❖ Both time and space efficiencies are measured as functions of the algorithm's input size.
- ❖ Time efficiency is measured by counting the number of times the algorithm's basic operation is executed.
- ❖ The efficiencies of some algorithms may differ significantly for inputs of the same size. For such algorithms, we need to distinguish between the worst-case, average-case, and best-case efficiencies.
- ❖ The framework's primary interest lies in the order of growth of the algorithm's running time as its input size goes to infinity.



Asymptotic notation

It is a way to describe the characteristics of a function in the limit.

It describes the rate of growth of functions.

Focus on what's important by abstracting away low-order terms and constant factors. It is a way to compare “sizes” of functions:

$$O \approx \leq \Omega \approx \geq \Theta \approx = o \approx < \omega \approx >$$





ASYMPTOTIC NOTATIONS

- The Efficiency analysis framework concentrates on the *order of growth of an algorithm's basic operation count* as the principal indicator of the algorithm's efficiency.
- To *Compare and rank the order of growth* of algorithm's basic operation count, computer scientists use three notations: *O (Big oh), Ω (Big omega), and Θ (Big theta)*.
- In the following discussion, $t(n)$ and $g(n)$ can be any two nonnegative functions defined on the set of natural numbers.
- *$t(n)$ will be an algorithm's running time* (usually indicated by its basic operation count $C(n)$), and $g(n)$ will be some simple function to compare the count with.



INFORMAL INTRODUCTION TO ASYMPTOTIC NOTATIONS

- Informally, $O(g(n))$ is the set of all functions with a *smaller or same order of growth* as $g(n)$ (to within a constant multiple, as n goes to infinity).

Examples:

❖ $n \in O(n^2)$, $100n + 5 \in O(n^2)$

The above two functions are *linear* and hence have a *smaller order of growth* than $g(n) = n^2$.

❖ $\frac{1}{2}n(n-1) \in O(n^2)$

The above function is *quadratic* and hence has the *same order of growth* as $g(n) = n^2$.

Note: $n^3 \notin O(n^2)$, $0.00001n^3 \notin O(n^2)$, $n^4 + n + 1 \notin O(n^2)$.

Functions n^3 and $0.00001n^3$ are both *cubic* and hence have a *higher order of growth than n^2* . Fourth-degree polynomial $n^4 + n + 1$ also has higher order of growth than n^2 .





INFORMAL INTRODUCTION TO ASYMPTOTIC NOTATIONS

- Informally, $\Omega(g(n))$ stands for the set of all functions with a *larger or same order of growth* as $g(n)$ (to within a constant multiple, as n goes to infinity).

Examples:

❖ $n^3 \in \Omega(n^2)$

The above function is *cubic* and hence has a *larger order of growth* than $g(n) = n^2$.

❖ $\frac{1}{2}n(n-1) \in \Omega(n^2)$

2

The above function is *quadratic* and hence has the *same order of growth* as $g(n) = n^2$.

Note: $100n + 5 \notin \Omega(n^2)$

Function $100n + 5$ is *linear* and hence has a *smaller order of growth* than n^2 .



INFORMAL INTRODUCTION TO ASYMPTOTIC NOTATIONS

- Informally, $\Theta(g(n))$ is the set of all functions that have the *same order of growth* as $g(n)$ (to within a constant multiple, as n goes to infinity).

Example:

- ❖ Every *quadratic function* $ax^2 + bx + c$ with $a > 0$ is in $\Theta(n^2)$.

Note:

- ❖ $100n + 5 \notin \Theta(n^2)$

Function $100n + 5$ is *linear* and hence has a *smaller order of growth than n^2* .

- ❖ $n^3 \notin \Theta(n^2)$

Function n^3 is *cubic* and hence has a *larger order of growth than n^2* .



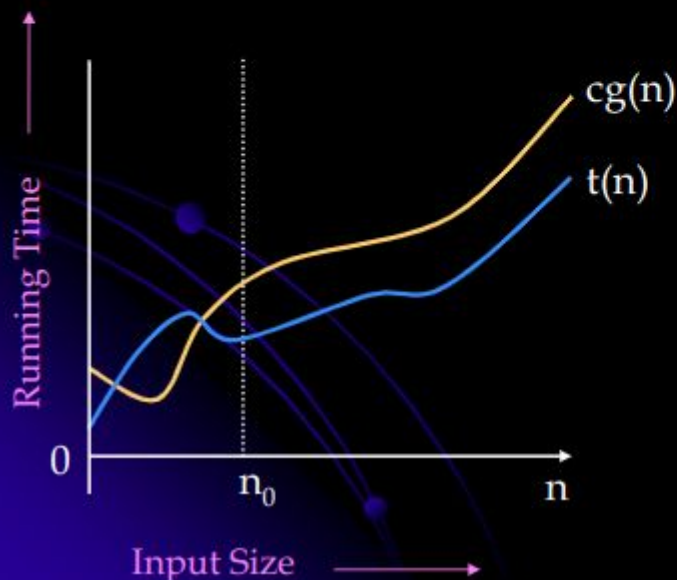


Formal Definition of Asymptotic Notations

O – Notation:

A function $t(n)$ is said to be in $O(g(n))$, denoted $t(n) \in O(g(n))$, if $t(n)$ is bounded above by some constant multiple of $g(n)$ for all large n , i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$$0 \leq t(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$



Notation: $t(n) = O(g(n))$





O – Notation:

➤ Example1:

$$2n + 10 \in O(n)$$

Proof: $2n + 10 \leq cn$

$$cn - 2n \geq 10$$

$$(c - 2)n \geq 10$$

$$n \geq 10/(c - 2)$$

Pick $c = 3$ and $n_0 = 10$

➤ Example2:

$$n^2 \notin O(n)$$

Proof: $n^2 \leq cn$

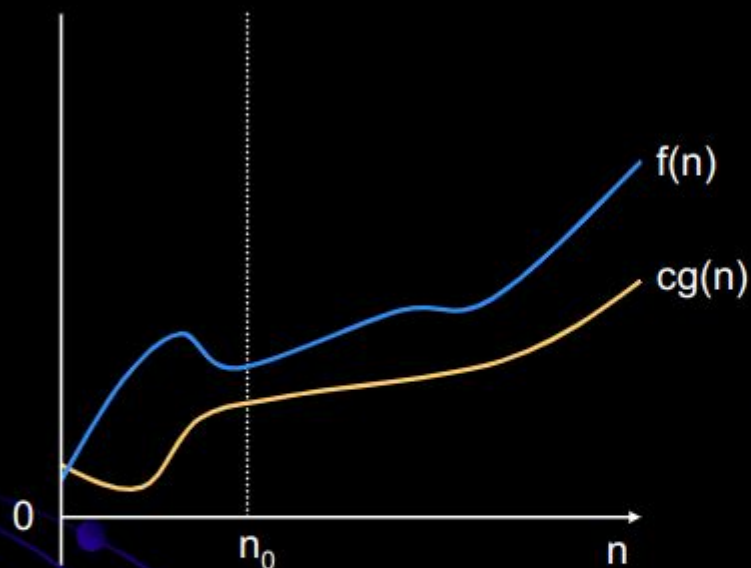
$$n \leq c$$

The above inequality cannot be satisfied since c must be a constant





Ω -notation



Notation: $f(n) = \Omega(g(n))$

■ $5n^2$ is $\Omega(n^2)$

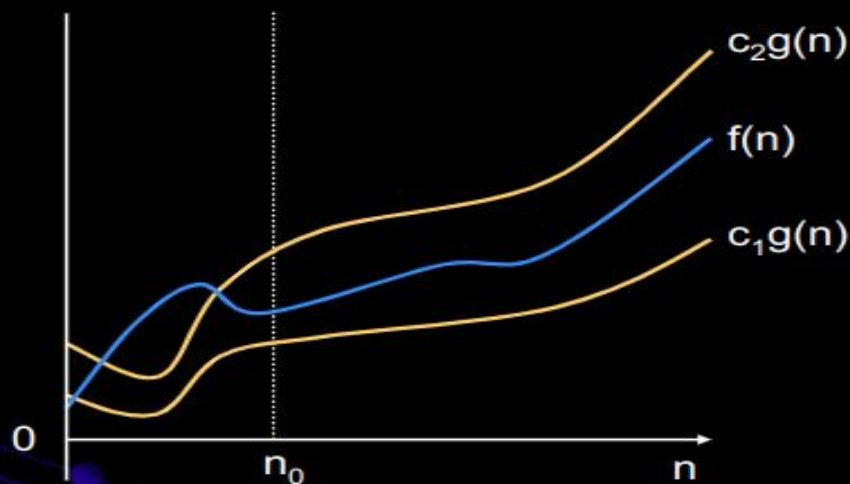
$f(n)$ is $\Omega(g(n))$ if there is a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq c \cdot g(n)$ for $n \geq n_0$

let $c = 5$ and $n_0 = 1$

30



Θ -notation



Notation: $f(n) = \Theta(g(n))$

$\Theta(g(n)) = \{f(n) :$

$\exists$ positive constants c_1, c_2 , and n_0 ,
such that $\forall n \geq n_0$,

we have $0 \leq c_1g(n) \leq f(n) \leq c_2g(n)$

$\}$



- **Algorithmic Complexity**
- The *algorithmic complexity* of a computation is, most generally, a measure of how *difficult* it is to perform the computation.
- That is, it measures some aspect of the **cost of computation** (in a general sense of “cost”).
 - Amount of resources required to do a computation.
- Some of the most common complexity measures:
 - “Time” complexity: # of operations or steps required
 - “Space” complexity: # of memory bits req’d

Algorithmic Complexity

- The *algorithmic complexity* of a computation is, most generally, a measure of how *difficult* it is to perform the computation.
- That is, it measures some aspect of the *cost* of computation (in a general sense of “cost”).
 - Amount of resources required to do a computation.
- Some of the most common complexity measures:
 - “Time” complexity: # of operations or steps required
 - “Space” complexity: # of memory bits required

Our focus

Complexity Depends on Input

- Most algorithms have different complexities for inputs of different sizes.
 - *E.g.* searching a long list typically takes more time than searching a short one.
- Therefore, complexity is usually expressed as a *function* of the input length.
 - This function usually gives the complexity for the *worst-case* input of any given length.

Example 1: Max algorithm

- **Problem:** Find the *simplest form* of the *exact* order of growth (Θ) of the *worst-case* time complexity (w.c.t.c.) of the *max* algorithm, assuming that each line of code takes some constant time every time it is executed (with possibly different times for different lines of code).

Complexity analysis of max

procedure *max*($a_1, a_2, \dots, a_n$: integers)

$v := a_1$

t_1

for $i := 2$ **to** n

t_2

if $a_i > v$ **then** $v := a_i$

t_3

return v

t_4

Times for *each*
execution of
each line.

First, what's an expression for the *exact* total worst-case time? (Not its order of growth.)

Complexity analysis, cont.

procedure $\text{max}(a_1, a_2, \dots, a_n : \text{integers})$

$v := a_1$

t_1

for $i := 2$ **to** n

t_2

if $a_i > v$ **then** $v := a_i$

t_3

return v

t_4

Times for *each*
execution of
each line.

w.c.t.c.:

$$t(n) = t_1 + \left(\sum_{i=2}^n (t_2 + t_3) \right) + t_4$$

Complexity analysis, cont.

Now, what is the simplest form of the exact (Θ) order of growth of $t(n)$?

The diagram illustrates the simplification of the recurrence relation $t(n) = t_1 + \left(\sum_{i=2}^n (t_2 + t_3) \right) + t_4$. Red arrows and circles highlight the following steps:

- t_1 is mapped to $\Theta(1)$.
- The summation term $\sum_{i=2}^n (t_2 + t_3)$ is mapped to $\sum_{i=2}^n \Theta(1)$.
- The term t_4 is mapped to $\Theta(1)$.
- The summation $\sum_{i=2}^n \Theta(1)$ is simplified to $(n-1)\Theta(1)$.
- The final expression $\Theta(1) + \Theta(n)\Theta(1)$ is simplified to $\Theta(1) + \Theta(n) = \Theta(n)$.

$$\begin{aligned} t(n) &= t_1 + \left(\sum_{i=2}^n (t_2 + t_3) \right) + t_4 \\ &= \Theta(1) + \left(\sum_{i=2}^n \Theta(1) \right) + \Theta(1) = \Theta(1) + (n-1)\Theta(1) \\ &= \Theta(1) + \Theta(n)\Theta(1) = \Theta(1) + \Theta(n) = \Theta(n) \end{aligned}$$



Mathematical Analysis of Nonrecursive Algorithms

General plan for Analyzing Time Efficiency of Nonrecursive Algorithms:

- Decide on a parameter(s) n indicating input's size.
- Identify algorithm's basic operation.
- Check whether the number of times the basic operation is executed depends only on the input size n . If it also depends on the type of input, investigate worst, average, and best case efficiency separately.
- Set up summation for $C(n)$ reflecting the number of times the algorithm's basic operation is executed.
- Simplify summation using standard formulas and establish the count's order of growth.





Two Basic Rules of Sum Manipulation :

$$\sum_{i=1}^u c a_i = c \sum_{i=1}^u a_i \quad (R_1)$$

$$\sum_{i=1}^u (a_i \pm b_i) = \sum_{i=1}^u a_i \pm \sum_{i=1}^u b_i \quad (R_2)$$

Two Summation Formulas :

$$\sum_{i=1}^u 1 = u - 1 + 1 \text{ where } 1 \leq u \text{ are some lower and upper integer limits} \quad (S_1)$$

$$\sum_{i=0}^n i = \sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2} \approx \frac{1}{2} n^2 \in \Theta(n^2) \quad (S_2)$$





Example 1: To find the largest element in a list of n numbers.

ALGORITHM *MaxElement*($A[0\dots n-1]$)

//Determines the value of the largest element in a given array

//Input: An array $A[0\dots n-1]$ of real numbers

//Output: The value of the largest element in A

maxval $\leftarrow A[0]$

for $i \leftarrow 1$ **to** $n-1$ **do**

if $A[i] > \text{maxval}$

maxval $\leftarrow A[i]$

return *maxval*





Algorithm analysis:

The measure of an input's size here is the number of elements in the array, i.e., n .

- There are two operations in the for loop's body:
 - The comparison $A[i] > \text{maxval}$ and
 - The assignment $\text{max val} \leftarrow A[i]$.
- The comparison operation is considered as the algorithm basic operation, because the comparison is executed on each repetition of the loop and not the assignment.
- The number of comparisons will be the same for all arrays of size n ; therefore, there is no need to distinguish among the worst, average, and best cases here.
- Let $C(n)$ denotes the number of times this comparison is executed. The algorithm makes one comparison on each execution of the loop, which is repeated for each value of the loop's variable i within the bounds 1 and $n - 1$, inclusive.

Therefore, the sum for $C(n)$ is calculated as follows:



Therefore, we get the following sum for $C(n)$:

$$C(n) = \sum_{i=1}^{n-1} 1.$$

This is an easy sum to compute because it is nothing other than 1 repeated $n - 1$ times. Thus,

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n).$$



Example 2: To Check whether all the elements in a given array are distinct or not.

ALGORITHM *UniqueElements*($A[0 \dots n-1]$)

//Determines whether all the elements in a given array are distinct

//Input: An array $A[0 \dots n-1]$

//Output: Returns “true” if all the elements in A are distinct

// and “false” otherwise

for $i \leftarrow 0$ to $n - 2$ do

 for $j \leftarrow i+1$ to $n - 1$ do

 if $A[i] = A[j]$ return false

return true





Algorithm analysis:

The measure of an input's size here is the number of elements in the array, i.e., n .

- There are two Comparisons:

one comparison is made for each repetition of the innermost loop, i.e., for each value of the loop variable j between its limits $i + 1$ and $n - 1$;

The same is repeated for each value of the outer loop, i.e., for each value of the loop variable i between its limits 0 and $n - 2$. Accordingly, we get the for loop's body:



Algorithm analysis:

$$\begin{aligned}C_{\text{worst}}(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) \\&= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2} \\&= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2).\end{aligned}$$

We also could have computed the sum $\sum_{i=0}^{n-2} (n-1-i)$ faster as follows:

$$\sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \dots + 1 = \frac{(n-1)n}{2},$$





MATHEMATICAL ANALYSIS FOR RECURSIVE ALGORITHMS

General Plan for Analyzing the Time Efficiency of Recursive Algorithms

1. Decide on a parameter (or parameters) indicating an input's size.
2. Identify the algorithm's basic operation.
3. Check whether the number of times the basic operation is executed can vary on different inputs of the same size; if it can, the worst-case, average-case, and best-case efficiencies must be investigated separately.
4. Set up a recurrence relation, with an appropriate initial condition, for the number of times the basic operation is executed.
5. Solve the recurrence or, at least, ascertain the order of growth of its solution.



Recursive Evaluation of $n!$ (factorial)

Definition: $n! = 1 * 2 * \dots * (n-1) * n$ for $n \geq 1$ and $0! = 1$

Recursive definition of $n!$: $F(n) = F(n-1) * n$ for $n \geq 1$ and $F(0) = 1$





Algorithm Analysis

ALGORITHM F(n)

//Computes n! recursively

//Input: A nonnegative integer n

//Output: The value of n!

if n = 0 return 1

else return F(n - 1)*n





Recursion





Objectives

become familiar with the idea of recursion
learn to use recursion as a programming tool
become familiar with the binary search algorithm as an example of recursion
become familiar with the merge sort algorithm as an example of recursion



How do you look up a name in the phone book?

One Possible Way

Search:

```
middle page = (first page + last page)/2
Go to middle page;
If (name is on middle page)
    done; //this is the base case
else if (name is alphabetically before middle page)
    last page = middle page //redefine search area to front half
    Search //same process on reduced number of pages
else //name must be after middle page
    first page = middle page //redefine search area to back half
    Search //same process on reduced number of pages
```

Overview

Recursion: a definition in terms of itself.

Recursion in algorithms:

- Natural approach to **some** (not all) problems
- A *recursive algorithm* uses itself to solve one or more smaller identical problems

Recursion in Java:

- Recursive methods implement recursive algorithms
- A *recursive method* includes a call to itself

Recursive Methods Must Eventually Terminate

*A recursive method must have
at least one base, or stopping, case.*

- A base case does not execute a recursive call
 - stops the recursion
- Each successive call to itself must be a "smaller version of itself"
 - an argument that describes a smaller problem
 - a base case is eventually reached

Key Components of a Recursive Algorithm Design

1. What is a smaller *identical* problem(s)?
 - Decomposition
2. How are the answers to smaller problems combined to form the answer to the larger problem?
 - Composition
3. Which is the smallest problem that can be solved easily (without further decomposition)?
 - Base/stopping case

Examples in Recursion

- Usually quite confusing the first time
- Start with some simple examples
 - recursive algorithms might not be best
- Later with inherently recursive algorithms
 - harder to implement otherwise

Factorial ($N!$)

- $N! = (N-1)! * N$ [for $N > 1$]
- $1! = 1$
- $3!$
 - $= 2! * 3$
 - $= (1! * 2) * 3$
 - $= 1 * 2 * 3$
- Recursive design:
 - Decomposition: $(N-1)!$
 - Composition: $* N$
 - Base case: $1!$

factorial Method

```
public static int factorial(int n)
{
    int fact;
    if (n > 1) // recursive case (decomposition)
        fact = factorial(n - 1) * n; // composition
    else // base case
        fact = 1;

    return fact;
}
```

```
public static int factorial(int 3)
{
    int fact;
    if (n > 1)
        fact = factorial(2) * 3;
    else
        fact = 1;
    return fact;
}
```

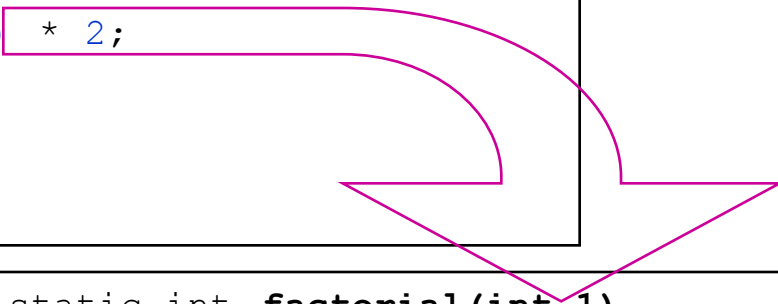


```
public static int factorial(int 3)
{
    int fact;
    if (n > 1)
        fact = factorial(2) * 3;
    else
        fact = 1;
    return fact;
}
```

```
public static int factorial(int 2)
{
    int fact;
    if (n > 1)
        fact = factorial(1) * 2;
    else
        fact = 1;
    return fact;
}
```

```
public static int factorial(int 3)
{
    int fact;
    if (n > 1)
        fact = factorial(2) * 3;
    else
        fact = 1;
    return fact;
}
```

```
public static int factorial(int 2)
{
    int fact;
    if (n > 1)
        fact = factorial(1) * 2;
    else
        fact = 1;
    return fact;
}
```



```
public static int factorial(int 1)
{
    int fact;
    if (n > 1)
        fact = factorial(n - 1) * n;
    else
        fact = 1;
    return fact;
}
```

```
public static int factorial(int 3)
{
    int fact;
    if (n > 1)
        fact = factorial(2) * 3;
    else
        fact = 1;
    return fact;
}
```

```
public static int factorial(int 2)
{
    int fact;
    if (n > 1)
        fact = factorial(1) * 2;
    else
        fact = 1;
    return fact;
}
```

```
public static int factorial(int 1)
{
    int fact;
    if (n > 1)
        fact = factorial(n - 1) * n;
    else
        fact = 1;
    return 1;
}
```

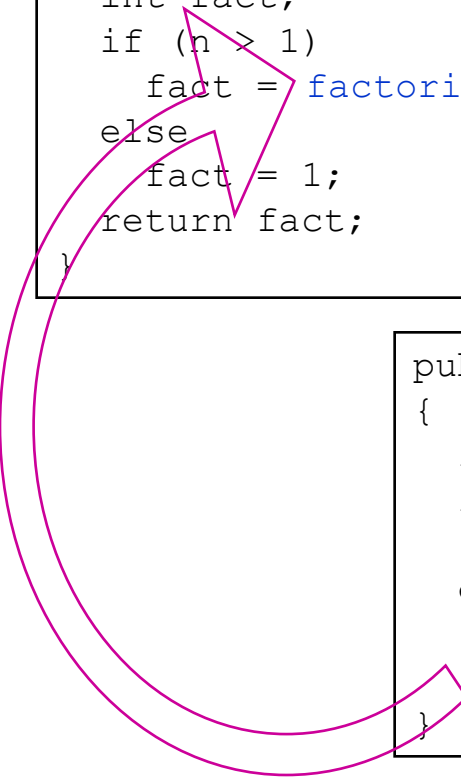
```
public static int factorial(int 3)
{
    int fact;
    if (n > 1)
        fact = factorial(2) * 3;
    else
        fact = 1;
    return fact;
}
```

```
public static int factorial(int 2)
{
    int fact;
    if (n > 1)
        fact = 1 * 2;
    else
        fact = 1;
    return fact;
}
```

```
public static int factorial(int 1)
{
    int fact;
    if (n > 1)
        fact = factorial(n - 1) * n;
    else
        fact = 1;
    return 1;
}
```

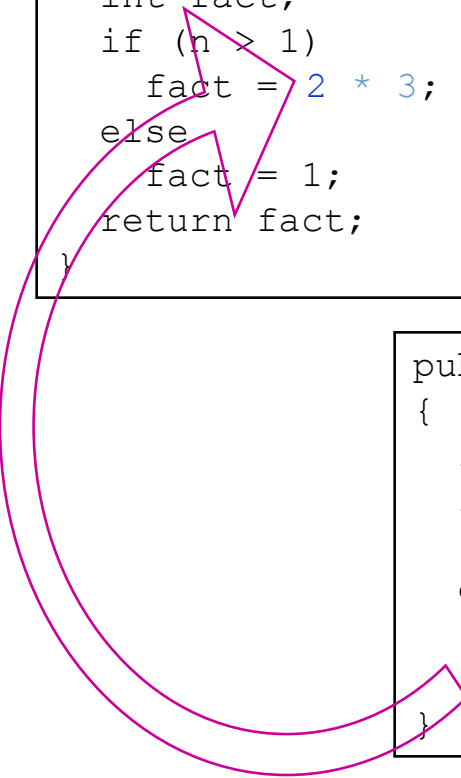


```
public static int factorial(int 3)
{
    int fact;
    if (n > 1)
        fact = factorial(2) * 3;
    else
        fact = 1;
    return fact;
}
```

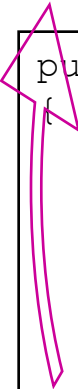


```
public static int factorial(int 2)
{
    int fact;
    if (n > 1)
        fact = 1 * 2;
    else
        fact = 1;
    return 2;
}
```

```
public static int factorial(int 3)
{
    int fact;
    if (n > 1)
        fact = 2 * 3;
    else
        fact = 1;
    return fact;
}
```



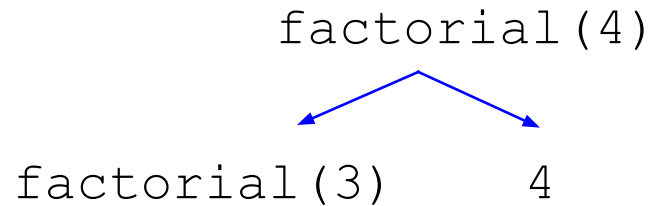
```
public static int factorial(int 2)
{
    int fact;
    if (n > 1)
        fact = 1 * 2;
    else
        fact = 1;
    return 2;
}
```



```
public static int factorial(int 3)
{
    int fact;
    if (n > 1)
        fact = 2 * 3;
    else
        fact = 1;
    return 6;
}
```

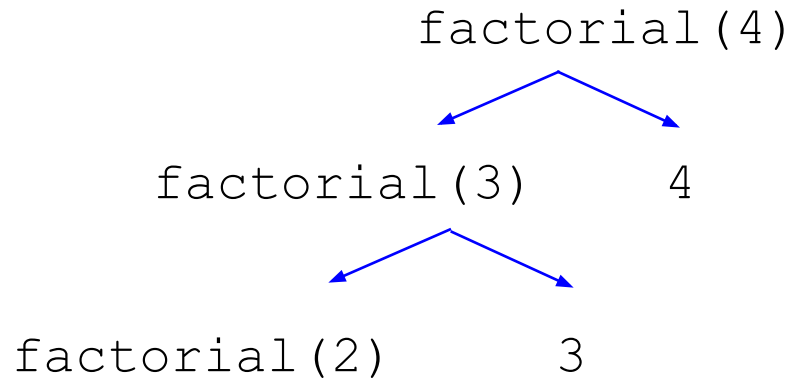
Execution Trace (decomposition)

```
public static int factorial(int n)
{
    int fact;
    if (n > 1) // recursive case (decomposition)
        fact = factorial(n - 1) * n; (composition)
    else // base case
        fact = 1;
    return fact;
}
```



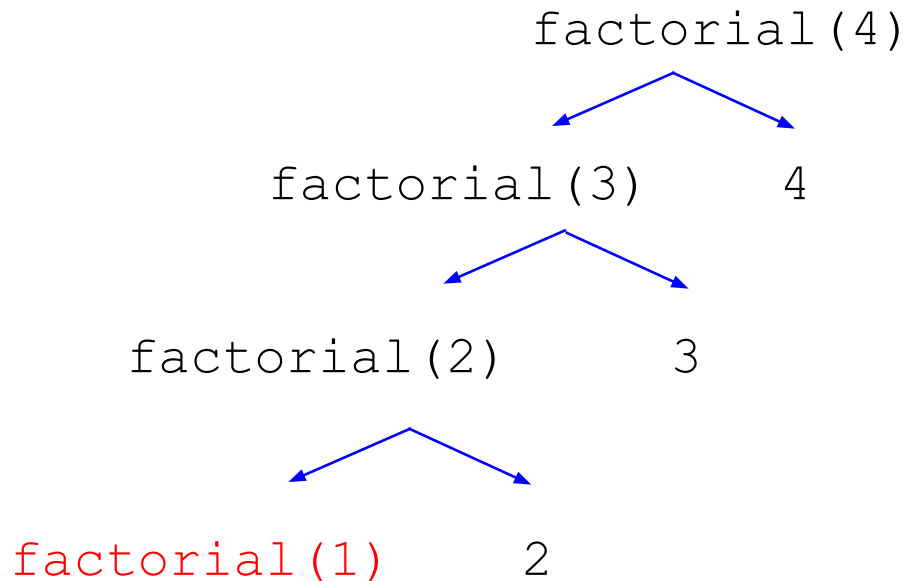
Execution Trace (decomposition)

```
public static int factorial(int n)
{
    int fact;
    if (n > 1) // recursive case (decomposition)
        fact = factorial(n - 1) * n; (composition)
    else // base case
        fact = 1;
    return fact;
}
```



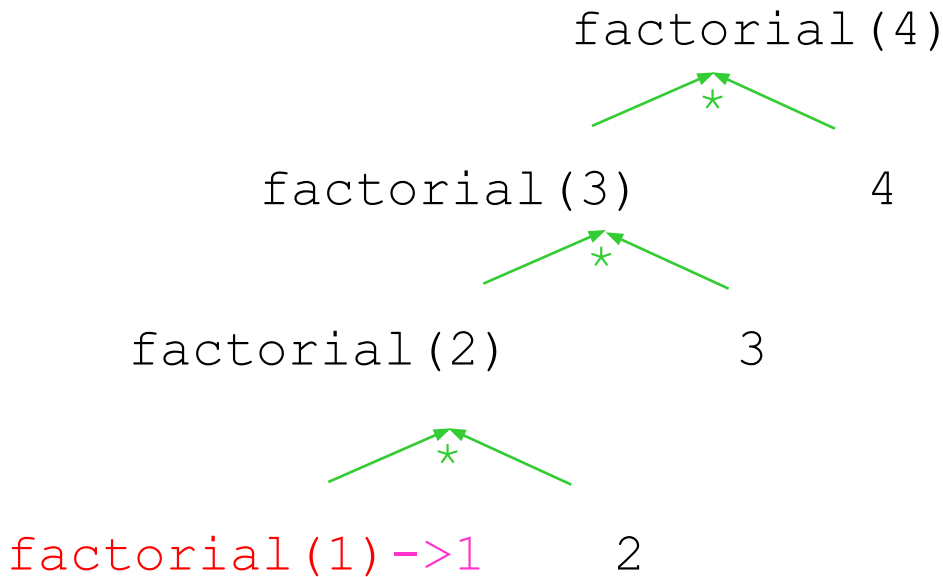
Execution Trace (decomposition)

```
public static int factorial(int n)
{
    int fact;
    if (n > 1) // recursive case (decomposition)
        fact = factorial(n - 1) * n; (composition)
    else // base case
        fact = 1;
    return fact;
}
```



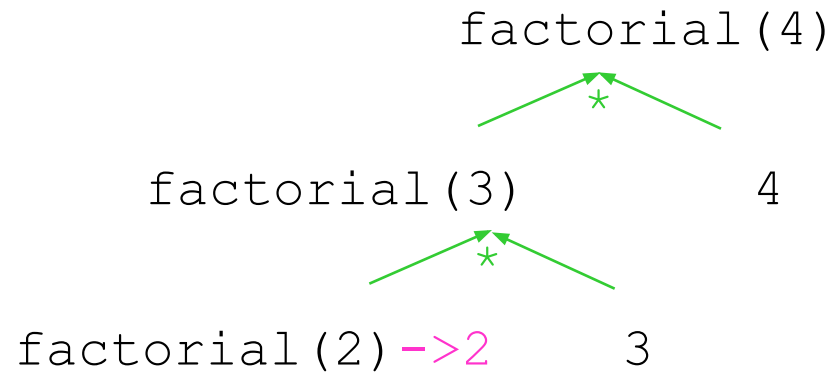
Execution Trace (composition)

```
public static int factorial(int n)
{
    int fact;
    if (n > 1) // recursive case (decomposition)
        fact = factorial(n - 1) * n; (composition)
    else // base case
        fact = 1;
    return fact;
}
```



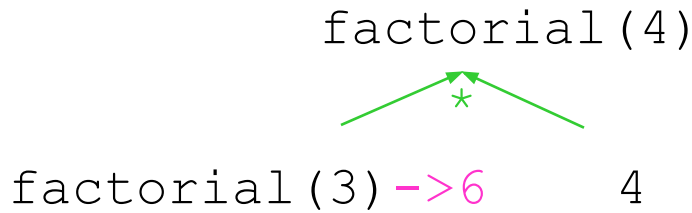
Execution Trace (composition)

```
public static int factorial(int n)
{
    int fact;
    if (n > 1) // recursive case (decomposition)
        fact = factorial(n - 1) * n; (composition)
    else // base case
        fact = 1;
    return fact;
}
```



Execution Trace (composition)

```
public static int factorial(int n)
{
    int fact;
    if (n > 1) // recursive case (decomposition)
        fact = factorial(n - 1) * n; (composition)
    else // base case
        fact = 1;
    return fact;
}
```



Execution Trace (composition)

```
public static int factorial(int n)
{
    int fact;
    if (n > 1) // recursive case (decomposition)
        fact = factorial(n - 1) * n; (composition)
    else // base case
        fact = 1;
    return fact;
}
```

factorial(4) -> 24

Improved factorial Method

```
public static int factorial(int n)
{
    int fact=1;    // base case value

    if (n > 1)    // recursive case (decomposition)
        fact = factorial(n - 1) * n; // composition
    // else do nothing; base case

    return fact;
}
```

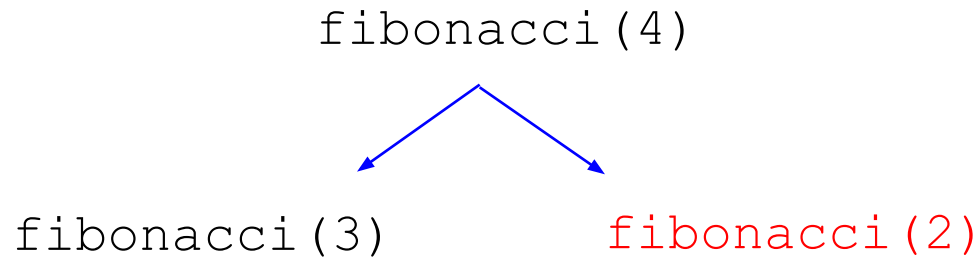
Fibonacci Numbers

- The N th Fibonacci number is the sum of the previous two Fibonacci numbers
- 0, 1, 1, 2, 3, 5, 8, 13, ...
- Recursive Design:
 - Decomposition & Composition
 - $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$
 - Base case:
 - $\text{fibonacci}(1) = 0$
 - $\text{fibonacci}(2) = 1$

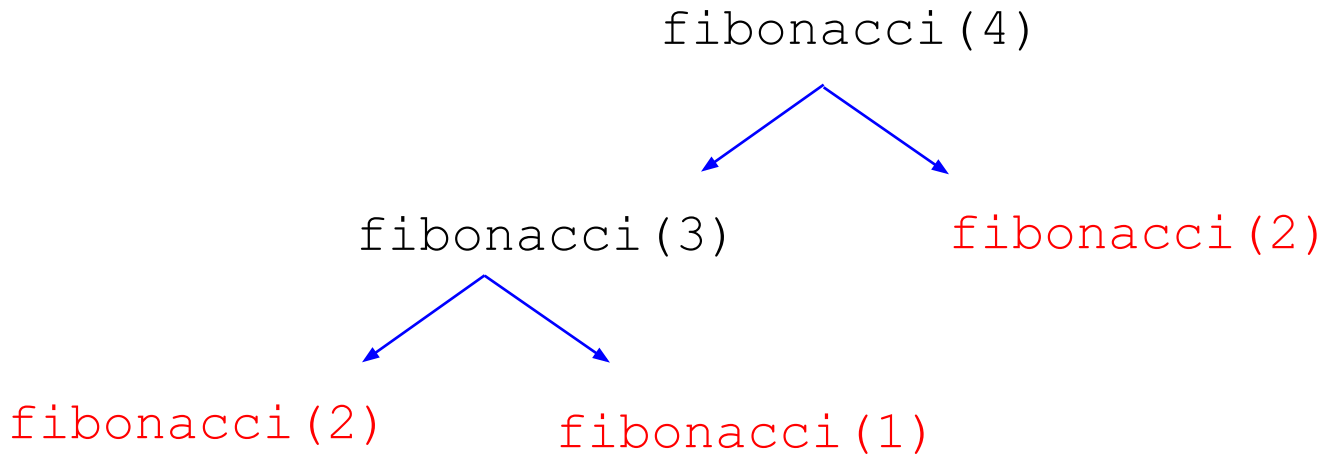
fibonacci Method

```
public static int fibonacci(int n)
{
    int fib;
    if (n > 2)
        fib = fibonacci(n-1) + fibonacci(n-2);
    else if (n == 2)
        fib = 1;
    else
        fib = 0;
    return fib;
}
```

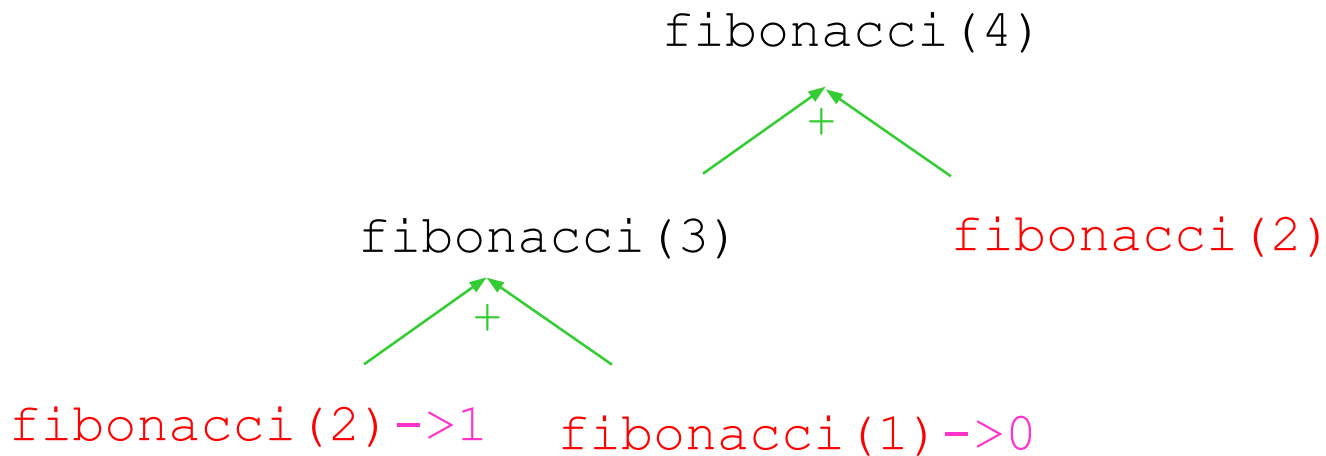
Execution Trace (decomposition)



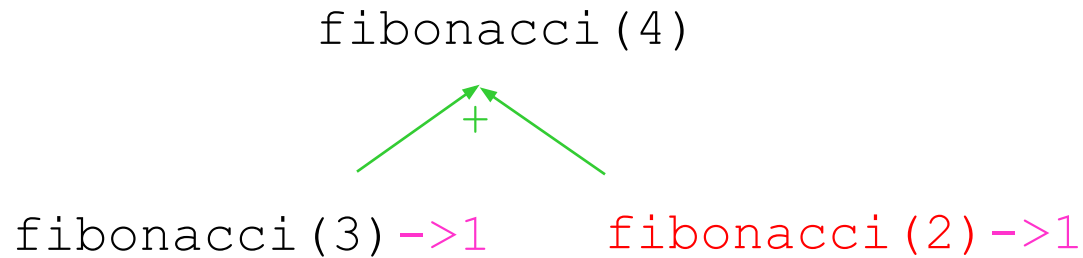
Execution Trace (decomposition)



Execution Trace (composition)



Execution Trace (composition)



Execution Trace (**composition**)

`fibonacci(4) -> 2`

Remember:

Key to Successful Recursion

- if-else statement (or some other branching statement)
- Some branches: recursive call
 - "smaller" arguments or solve "smaller" versions of the same task (*decomposition*)
 - Combine the results (*composition*) [if necessary]
- Other branches: no recursive calls
 - *stopping* cases or *base* cases

Template

```
... method (...)  
{  
    if ( ... ) // base case  
    {  
    }  
    else // decomposition & composition  
    {  
    }  
    return ... ; // if not void method  
}
```


Template (only one base case)

```
... method(...)
{
    ... result = ...    ; //base case

    if ( ... ) // not base case
    { //decomposition & composition
        result = ...
    }

    return result;
}
```

Summary

- *Recursive call*: a method that calls itself
- Powerful for algorithm design at times
- Recursive algorithm design:
 - **Decomposition** (smaller identical problems)
 - **Composition** (combine results)
 - **Base case(s)** (smallest problem, no recursive calls)
- Implementation
 - Conditional (e.g. if) statements to separate different cases
 - Avoid infinite recursion
 - Problem is getting smaller (decomposition)
 - Base case exists and reachable
 - Composition could be tricky



Algorithm Analysis

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0$$

to compute $F(n-1)$ to multiply $F(n-1)$ by n

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0$$

$$M(0) = 0$$

$$M(0) = 0$$

the calls stop when $n=0$ no multiplications when $n = 0$





Algorithm Analysis

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0$$

to compute $F(n-1)$ to multiply $F(n-1)$ by n

$$M(n) = M(n - 1) + 1 \quad \text{for } n > 0$$

$$M(0) = 0$$

$$M(0) = 0$$

the calls stop when $n=0$ no multiplications when $n = 0$





Algorithm Analysis using back substitution method

$$M(n) = M(n - 1) + 1$$

$$M(n-1) = M(n-2) + 1; \quad // \text{substitute } M(n - 1) = M(n - 2) + 1$$

$$M(n-2) = M(n-3) + 1 \quad // \text{substitute } M(n - 2) = M(n - 3) + 1$$

$$M(n) = [M(n-2) + 1] + 1$$

$$= M(n-2) + 2$$

$$= [M(n-3) + 1] + 2$$

$$= M(n-3) + 3$$

$$= M(n-n) + n$$

$$= n$$

Overall time Complexity: $O(n)$





Master Theorem: Solving recurrence relations using master theorem

MASTER THEOREM

To solve a recurrence relation running time you can use many different techniques. One popular technique is to use the Master Theorem also known as the Master Method. “ In the analysis of algorithms, the master theorem provides a solution in asymptotic terms (using Big O notation) for recurrence relations of types that occur in the analysis of many divide and conquer algorithms.



Master Theorem: Solving recurrence relations using master theorem

MASTER THEOREM

To solve a recurrence relation running time you can use many different techniques. One popular technique is to use the Master Theorem also known as the Master Method. “ In the analysis of algorithms, the master theorem provides a solution in asymptotic terms (using Big O notation) for recurrence relations of types that occur in the analysis of many divide and conquer algorithms.



Master Theorem: Solving recurrence relations using master theorem

A Relation is Said to be Recurrence relation if it is in the following form

$$T(n) = a T(n/b) + f(n)$$

In such cases As per master theorem:

- 1) If $a > b^k$, then $T(n) = \Theta(n^{\log_b^a})$
- 2) If $a = b^k$
 - a. If $p > -1$, then $T(n) = \Theta(n^{\log_b^a} \log^{p+1} n)$
 - b. If $p = -1$, then $T(n) = \Theta(n^{\log_b^a} \log \log n)$
 - c. If $p < -1$, then $T(n) = \Theta(n^{\log_b^a})$
- 3) If $a < b^k$
 - a. If $p \geq 0$, then $T(n) = \Theta(n^k \log^p n)$
 - b. If $p < 0$, then $T(n) = O(n^k)$





Master Theorem: Solving recurrence relations using master theorem

For each of the following recurrences, give an expression for the runtime $T(n)$ if the recurrence can be solved with the Master Theorem. Otherwise, indicate that the Master Theorem does not apply.

Problem-1 $T(n) = 3T(n/2) + n^2$

Solution: $T(n) = 3T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2)$ (Master Theorem Case 3.a)

Problem-2 $T(n) = 4T(n/2) + n^2$

Solution: $T(n) = 4T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2 \log n)$ (Master Theorem Case 2.a)



1) Problem: $T(n) = 3T(n/2) + n^2$

Solution: $T(n) = \Theta(n^2)$

2) Problem: $T(n) = 4T(n/2) + n^2$

Solution: $T(n) = \Theta(n^2 \log n)$

3) Problem: $T(n) = T(n/2) + 2n$

Solution: $\Theta(2n)$

